



Tribhuvan University

Faculty of Humanities and Social Sciences

RESTRONET (A Restaurant Recommendation System)

A PROJECT REPORT

Submitted to

Department of Computer Application

Kathmandu Model College

*In partial fulfillment of the requirements for the Bachelors in Computer
Application*

Submitted by:

Ashok Rai

TU Reg. No: 6-2-446-9-2020

BCA 8th Semester

Shraddha Dongol

TU Reg. No: 6-2-446-28-2020

BCA 8th Semester

Under the Supervision of

Ms. Prakriti Shree Tuladhar



Tribhuvan University

Faculty of Humanities and Social Sciences

Kathmandu Model College

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this internship report is prepared under my supervision by **ASHOK RAI (6-2-446-9-2020)** and **SHRADDHA DONGOL (6-2-446-28-2020)** entitled "**RESTRONET - A RESTAURANT RECOMMENDATION SYSTEM**" in the partial fulfillment of the requirements for the degree of Bachelor of Computer Application is recommended for the final evaluation.

.....

Ms. Prakriti Shree Tuladhar

SUPERVISOR

Department of Computer Application

Kathmandu Model College



Tribhuvan University

Faculty of Humanities and Social Sciences

Kathmandu Model College

LETTER OF APPROVAL

This is to certify that this project is prepared by **ASHOK RAI (6-2-446-28-2020)** and **SHRADDHA DONGOL (6-2-446-28-2020)** entitled “**RESTRONET - A RESTAURANT RECOMMENDATION SYSTEM**” in the partial fulfillment of the requirement for the degree of Bachelor of Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

..... Ms.Prakriti Shree Tuladhar Project Supervisor Kathmandu Model College	 Mr. Binod Poudyal Project Coordinator Kathmandu Model College
..... External Examiner Tribhuvan University	 Internal Examiner Tribhuvan University

ACKNOWLEDGEMENT

We would like to express our sincere appreciation to everyone who has been involved in this project from its inception. Writing this project has been one of the most significant academic challenges we have faced, and without the support, patience, and guidance of those involved, it would not have been completed. We are deeply grateful to them.

It is with great pleasure that we present this project report on the "Restaurant Recommendation System." We consider it a privilege to have worked with a dedicated team of project guides who have supported us throughout the course of this project. The success of this endeavor is the result of their hard work, commitment, and invaluable assistance.

We would also like to thank Kathmandu Model College for providing the opportunity, resources, and a conducive environment for learning and research, which helped us complete our final semester project. Our gratitude extends to our supervisor, Ms Prakriti Shree Tuladhar, and our coordinator, Mr. Binod Poudyal , for their guidance throughout the research and development of this project. We are also thankful to our friends who assisted us with the challenges we encountered and offered the right advice. Special thanks to everyone involved in this project—this achievement would not have been possible without their contributions.

Yours sincerely,

Ashok Rai

Shraddha Dongol

ABSTRACT

The main objective of this report is to highlight the development and practical implementation of the Restaurant Recommendation System. The system aims to address the challenges users face when searching for suitable restaurants based on their preferences, such as cuisine type, location, price range, and ratings. With the growing number of dining options available, it is essential to provide a platform that offers accurate recommendations, reliable reviews, and a smooth user experience. This system integrates real-time data to display restaurant information, user-generated reviews, and overall ratings, enabling users to make informed dining decisions.

A dedicated superadmin module is included to efficiently manage restaurant details, such as menu updates, offers, photos, and basic information. The superadmin can also oversee user activities, handle reported reviews, and ensure that the platform maintains quality and authenticity. The application is built using modern technologies to ensure secure data handling, fast response times, and easy accessibility. This report covers the design, development, and testing of the system, along with the challenges encountered and the solutions implemented to create a robust and user-friendly restaurant recommendation platform.

Keywords: *Restaurant Recommendation System, User Reviews, Real-Time Data, Superadmin Dashboard, User Preferences.*

TABLE OF CONTENTS

SUPERVISOR’S RECOMMENDATION	I
LETTER OF APPROVAL	II
ACKNOWLEDGEMENT	III
ABSTRACT	IV
LIST OF ABBREVIATIONS	VII
LIST OF FIGURES	VIII
LIST OF TABLES	IX
CHAPTER 1: INTRODUCTION	1
1.1 Introduction	1
1.2 Problem Statement	1
1.3 Objectives	1
1.4 Scope and Limitations	1
1.5 Development methodology	2
1.6 Report Organization	2
CHAPTER 2 - BACKGROUND STUDY AND LITERATURE REVIEW	4
2.1 Background Study	4
2.2 Literature Review	5
CHAPTER 3 - SYSTEM ANALYSIS AND DESIGN	7
3.1 System Analysis	8
3.1.1 Requirement Analysis	8
I. Functional Requirements	8
II. Non Functional Requirements	9
3.1.2 Feasibility Analysis:	10
1. Technical Feasibility:	10
2. Economic Feasibility:	10
3. Operational Feasibility:	11
4. Legal and Regulatory Feasibility:	11
3.1.3 Data Modeling (ER Diagram)	11
3.1.4 Dynamic Modeling using Sequence Diagram	12
3.1.5 Process Modeling	12
3.2 System Design	13

3.2.1 System Architecture	13
3.2.2 Database Schema Design	14
3.2.3 System Flowchart	14
3.3 Algorithm Details	15
CHAPTER 4 - IMPLEMENTATION AND TESTING	20
4.1 Implementation	21
4.1.1 Tools used	21
4.1.2 Implementation details of modules	22
4.2 Testing	26
4.2.1 Test cases for Unit Testing	27
4.2.2 Test cases for System Testing	27
CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS	31
5.1 Conclusion	31
5.2 Future Recommendation	31
REFERENCE	32
APPENDIX	33

LIST OF ABBREVIATIONS

API	Application Programming Interface
CBF	Content-Based Filtering
CNN	Convolutional Neural Networks
CSS	Cascading Style Sheet
DB	Database
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
MVC	Model View Architecture
TF	Term Frequency
IDF	Inverse Document Frequency
UI	User Interface
UX	User Experience

LIST OF FIGURES

Figure. 1. Agile Methodology	2
Figure 2 :Use Case Diagram for Restaurant Recommendation System	9
Figure 3 : Entity Relationship Diagram for Restaurant Recommendation System	11
Figure 4 : Sequence Diagram for Recommendation System	12
Figure 5 : DFD Level 0 for Restaurant Recommendation System	12
Figure 6 : DFD Level 1 for Restaurant Recommendation System	13
Figure 7 : Client Server Architecture	13
Figure. 8. Database Schema Design	14
Figure. 9. Flow chart of Superadmin	15
Figure. 10. Flow chart of User	15

LIST OF TABLES

Table 1 : Functional Requirements	8
Table 2 : Test case for User Registraion.....	22
Table 3 : Test case for User Login.....	23
Table 4 : Test case for Filtering Restaurants	28
Table 5 : Test case for Managing Restaurants	29
Table 6 : Test case for Managing Menu	30

CHAPTER 1: INTRODUCTION

1.1 Introduction

Modern consumers are not just looking for restaurants to dine at, they seek personalized experiences like birthday celebration venues, date spots, or places that fit specific cuisine preferences and locations. A general list of restaurants no longer satisfies the demand for tailored recommendations. This project introduces a restaurant and occasion-based recommendation system using React and NodeJs, aimed at providing event and mood specific suggestions for users based on their needs, preferences, and location.

Visitors can browse all restaurants in Local Area, and get information about restaurant name, address , phone, rating , price, and map. The functions include searching restaurants, viewing/giving rating to restaurants, viewing their menu items and so on.

1.2 Problem Statement

- There are too many restaurant options with no tailored recommendations.
- Users waste time browsing and still end up with unsatisfactory choices.
- Existing platforms don't consider:
 - Type of event or celebration
 - Preferred cuisines or themes
 - Atmosphere or vibe

1.3 Objectives

- To build a web application that recommends venues based on occasion (e.g., gatherings, birthday), cuisine, ambiance, and location.
- To create an intuitive interface for users to search, filter, and receive personalized suggestions.
- To allow super admin-side management for restaurant/event listings and categorization.

1.4 Scope and Limitations

Scope

- Suggest restaurants for various experiences like birthdays, family gathering, or casual hangouts.
- Location-based filtering using React Leaflet.

- Filter by cuisine, categories, tags, ratings, and user preferences.
- Superadmin dashboard for adding/updating restaurant and venue details.

Limitations

- Recommendations depend on available data (may not cover all regions initially).
- Real-time user feedback may not be immediately reflected in suggestions.
- User ratings and reviews may be biased, outdated, or unreliable.
- The system may face a cold-start problem for new users or newly added restaurants due to limited interaction data.

1.5 Development methodology

At the start of the project, the requirements were gathered and reviewed, allowing for flexibility in adapting to new ideas and changes during development. The Agile methodology was chosen to enable iterative progress, with the project broken into smaller tasks and features. Each phase focused on building and testing specific functionalities, ensuring continuous improvement and refinement throughout the process. Regular reviews allowed for adjustments based on feedback, helping to integrate new features and enhance the system as the project evolved.

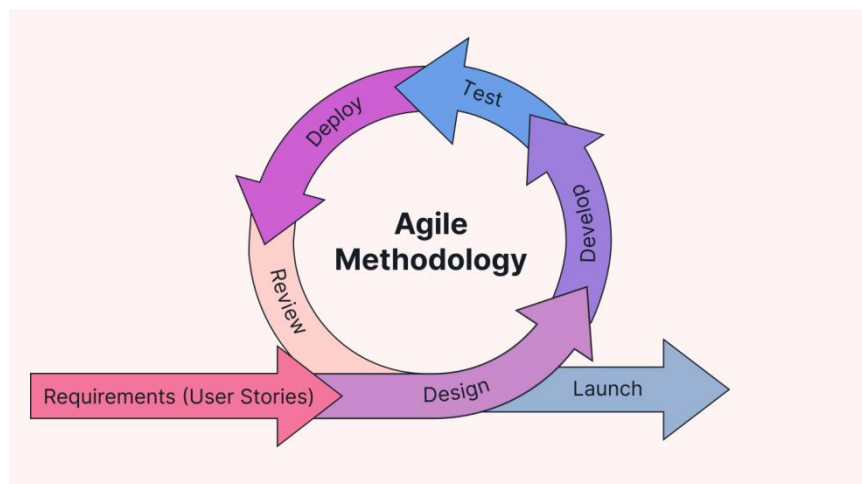


Figure. 1. Agile Methodology

1.6 Report Organization

The report is organized into five detailed chapters as follows:

Chapter 1: Introduction

This chapter introduces the project by outlining the problem statement, defining the objectives, and explaining the overall scope and limitations of the project. It provides an overview of the project's significance and sets the foundation for the subsequent chapters, helping readers understand the goals and constraints within which the project operates.

Chapter 2: Background and Literature Review

Chapter 2 presents an in-depth exploration of the background study and literature review related to the project. It examines existing research, relevant theories, and previous work in the field, providing context for the project. This chapter highlights key findings, methodologies, and frameworks that inform the project's design and implementation.

Chapter 3: System Analysis and Design

This chapter delves into the analysis of the current system, identifying its strengths, weaknesses, and areas for improvement. It also covers the system analysis and design of the proposed solution, detailing the approaches, tools, and techniques used to develop the new system. This chapter emphasizes the requirements gathering, system architecture, and design considerations that guide the project's development.

Chapter 4: Implementation and Testing

Chapter 4 focuses on the implementation phase of the project, describing the technical processes involved in developing and deploying the system. It also covers the testing phase, where various tests were conducted to evaluate the system's performance, functionality, and reliability. This chapter highlights key challenges faced during implementation and the solutions applied to ensure the project's success.

Chapter 5: Conclusion and Lessons Learned

The final chapter provides a summary of the project, reflecting on the achievements, challenges, and key takeaways. It offers a conclusion based on the outcomes of the project and discusses the lessons learned throughout the process. Additionally, this chapter suggests possible future improvements and directions for further research or development related to the project.

CHAPTER 2 - BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Restaurant recommendation systems have become an integral part of modern day urban living, particularly with the growth of mobile applications and location-based services. Several studies and existing applications have addressed this problem using different technologies and approaches, ranging from collaborative filtering to content-based and hybrid models, often enhanced with machine learning and user behavior analysis.

A popular commercial example is Zomato, which recommends restaurants based on user preferences, location, and reviews. It uses collaborative filtering and sentiment analysis on user reviews to provide personalized recommendations. Similarly, Yelp uses a combination of user-generated reviews, star ratings, and geolocation to help users find local businesses, including restaurants. These systems heavily rely on user feedback and community participation for effectiveness [1].

In academic research, Park et al. (2008) proposed a restaurant recommendation system using collaborative filtering techniques to analyze user behavior and preferences, showing that filtering methods can effectively improve the relevance of recommendations [2]. However, collaborative filtering suffers from cold start problems where recommendations are less effective for new users or newly added restaurants.

To overcome these challenges, Zheng et al. (2010) introduced a location-based restaurant recommendation system that incorporates geospatial data into collaborative filtering algorithms to improve context-awareness. Their work highlighted the importance of integrating physical proximity with user preference to enhance recommendation accuracy [3].

A more advanced approach was proposed by Liu et al. (2014), who applied context-aware hybrid recommendation systems for mobile users. Their model used GPS data, time, and user mood to personalize recommendations, making it especially relevant for use cases such as recommending restaurants for specific events like birthdays or dates [4].

Recent developments include the use of deep learning for natural language processing of restaurant reviews and image-based recommendations using food photos and menus. For example, Chen et al. (2018) developed a deep neural network that combines user preferences, image recognition, and restaurant metadata to generate highly accurate suggestions [5].

These studies and systems collectively demonstrate the evolving nature of restaurant recommendation systems, highlighting the need for hybrid models that combine content, collaborative, and context-based filtering. For our system, we aim to incorporate event-specific recommendations (e.g., birthday, date), cuisine preferences, and real-time geolocation using a user-friendly React frontend and Node.js backend, enhancing the personalized experience beyond simple search and filter functions.

2.2 Literature Review

The growth of recommendation systems in the food and hospitality domain has motivated extensive research on techniques that improve personalization, accuracy, and contextual relevance. Early recommendation systems primarily relied on collaborative filtering, which analyzes user-item interactions to predict preferences. Park et al. (2008) demonstrated that CF can significantly improve restaurant recommendation accuracy by leveraging similarities among users with shared tastes [2]. Despite its popularity, CF faces inherent challenges such as data sparsity and the cold start problem, where insufficient information about new users or restaurants degrades system performance.

To mitigate these limitations, researchers explored content-based filtering (CBF), which recommends restaurants by comparing item attributes such as cuisine type, dining style, or features like ambience and price range with user interests. While CBF effectively handles new restaurants, its recommendations tend to be narrow, often lacking diversity because they rely solely on previously liked attributes.

Recognizing the limitations of single-method approaches, several studies have proposed hybrid recommendation systems that combine collaborative and content-based techniques. Liu et al. (2014) presented a context-aware hybrid model that incorporates additional contextual data such as time of day, weather, and user mood to provide highly adaptive

restaurant suggestions for mobile environments [4]. Their findings indicate that integrating contextual signals significantly enhances recommendation relevance, particularly in dynamic real-world scenarios.

Another major advancement in the literature concerns the integration of location-based data. Zheng et al. (2010) introduced a location-enhanced collaborative filtering technique that incorporates spatial proximity into the recommendation algorithm [3]. This approach acknowledges that user decisions about dining are strongly influenced by geographic convenience. Studies that followed further emphasized geolocation as a critical contextual factor, especially in mobile recommendation systems where real-time positioning is available.

In recent years, research has increasingly shifted toward deep learning-based approaches, leveraging neural networks to extract richer semantic information from user reviews, images, menus, and behavioral patterns. Chen et al. (2018) demonstrated how combining convolutional neural networks (CNNs) with user preference modeling can provide highly personalized recommendations that surpass traditional heuristic methods [5]. Deep learning enables the modeling of complex interactions between textual sentiments, visual cues, and restaurant metadata, offering multi-modal insights that static methods struggle to achieve. Commercial platforms such as Zomato and Yelp have applied many of these techniques in real-world deployments. Their systems rely heavily on user-generated content—reviews, ratings, photos—to drive machine learning models that recommend restaurants based on sentiment analysis, popularity trends, and behavioral analytics [1]. These platforms highlight the importance of large-scale data, community participation, and continuous model refinement for maintaining recommendation quality.

Across the literature, a consistent theme emerges: effective restaurant recommendation requires a multi-dimensional approach, combining traditional preference-based filtering with contextual inputs such as location, time, purpose of visit, and user behavior. However, relatively fewer studies have focused on event-specific recommendations—such as dining for birthdays, dates, family gatherings, or business meetings—which represents a promising area for innovation. The literature suggests that incorporating hybrid filtering, contextual awareness, and geospatial intelligence can substantially improve personalization, forming a strong foundation for the system proposed in this project.

Yelp has been widely referenced in academic research as a large-scale, real-world platform for evaluating restaurant recommendation systems. The Yelp platform leverages user-generated content such as ratings, textual reviews, photos, and check-in data to generate personalized recommendations. Several studies utilizing the Yelp Dataset have demonstrated the effectiveness of combining collaborative filtering with content-based and sentiment-aware approaches. By applying natural language processing techniques to analyze review texts, Yelp-style systems are able to extract sentiments related to food quality, service, ambience, and price, thereby improving recommendation accuracy [6]. In addition, Yelp integrates location-based filtering to prioritize nearby restaurants, recognizing geographic proximity as a key factor in dining decisions. Despite its effectiveness, existing Yelp-based recommendation approaches face challenges including review bias, data sparsity for new users (user cold-start problem), and limited support for occasion- or event-specific recommendations, highlighting gaps that motivate further research in personalized and context-aware restaurant recommendation systems [6].

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

3.1.1 Requirement Analysis

I. Functional Requirements

Table 1 : Functional Requirements

S.No.	Requirement	Priority
1	Shall allow users to search restaurants based on location, cuisine, and meal-types(lunch, brunch, dinner)	Must have
2	Shall provide restaurant recommendations with details like name, address, cuisine, and ratings	Must have
3	Shall allow filtering and sorting of results based on rating, price, and distance	Should have
4	Shall display restaurant locations on a map with markers	Must have
5	Shall allow users to view restaurant details in a separate page	Must have
6	Shall allow superadmin to add, update, or remove restaurant data	Must have
7	Shall allow users to give feedback or rating to improve future recommendations	Should have

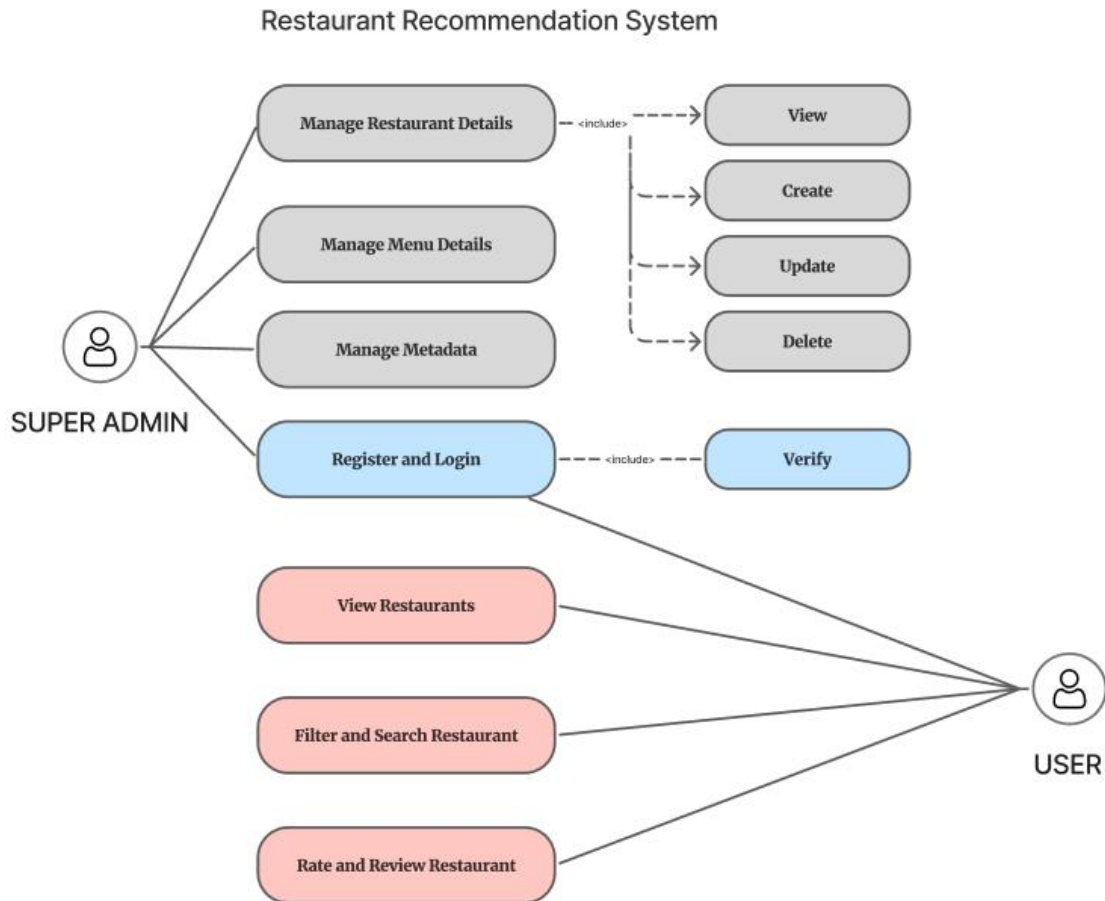


Figure. 2. Use Case Diagram for Restaurant Recommendation System

II. Non Functional Requirements

- **System Responsiveness**

The system shall respond to user actions, such as searching or filtering restaurants, within 2–3 seconds to ensure a smooth user experience.

- **Data Security**

User information such as login credentials and saved preferences must be securely stored using encryption to protect privacy and prevent misuse.

- **User-Friendly Interface**

The platform must have a clean, intuitive UI that enables users to navigate easily through restaurant listings, filters, and event categories.

- **Scalability and Flexibility**

The system should be designed to accommodate future feature additions like AI-based personalization or table reservation integration.

- **Accessibility**

The application should provide features like keyboard navigation, readable fonts, and screen reader support to ensure accessibility for all users.

3.1.2 Feasibility Analysis:

Feasibility analysis assesses whether the system can be realistically implemented, given the technical, financial, and operational constraints.

1. Technical Feasibility:

- **Existing Technology:** Technologies such as Google Maps API, React.js for frontend, and Node.js with Express for backend have been widely adopted for similar use cases globally. This ensures strong community support and proven implementation patterns.
- **Infrastructure:** Since the system will be web-based, it requires stable internet access. In urban areas where restaurants are concentrated, this is typically not a barrier.
- **Integration:** The system can integrate third-party APIs for maps, reviews, or reservation platforms (e.g., Zomato, Yelp, or Google Places) to enhance recommendation capabilities.

2. Economic Feasibility:

- **Costs:** Initial development costs will include UI/UX design, API integrations, server hosting, and testing. However, using open-source tools (e.g., React, Node.js, MongoDB) reduces licensing costs. Potential monetization through restaurant partnerships or ads could offset operational costs.
- The system is developed using open-source technologies such as React, Node.js, MongoDB, and Leaflet.js, which significantly reduces development and licensing costs. Leaflet.js, combined with free map services such as OpenStreetMap, eliminates dependency on paid mapping APIs.

Operational costs mainly include server hosting and runtime expenses for the backend and database. Since free mapping services are used, there are no direct API usage costs at the current scale. These costs remain manageable for small- to medium-scale deployment. In the future, monetization through restaurant partnerships, featured listings, or advertisements could help offset operational expenses and ensure sustainability.

3. Operational Feasibility:

- **User Training:** The system is intended for public use, so minimal training is required. An intuitive design and onboarding walkthroughs will support first-time users.
- **Maintenance:** Regular updates will be scheduled to improve algorithms, fix bugs, and add features. A dedicated technical team will handle ongoing maintenance and support.

4. Legal and Regulatory Feasibility:

- **Compliance:** The restaurant recommendation system must comply with Nepal's data privacy and electronic transaction regulations, such as the Privacy Act, 2018 and the Electronic Transactions Act, 2008, ensuring that user data is collected, stored, and processed responsibly. Additionally, the system should maintain transparency and fairness in restaurant recommendations, avoiding bias or manipulation to ensure equal and ethical representation of local restaurants.

3.1.3 Data Modeling (ER Diagram)

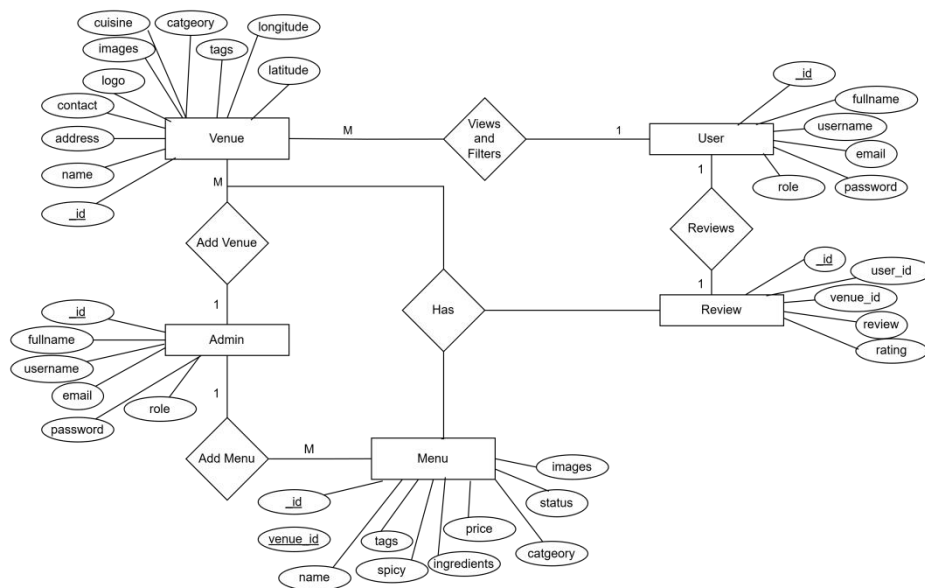


Figure 3 : Entity Relationship Diagram for Restaurant Recommendation System

3.1.4 Dynamic Modeling using Sequence Diagram

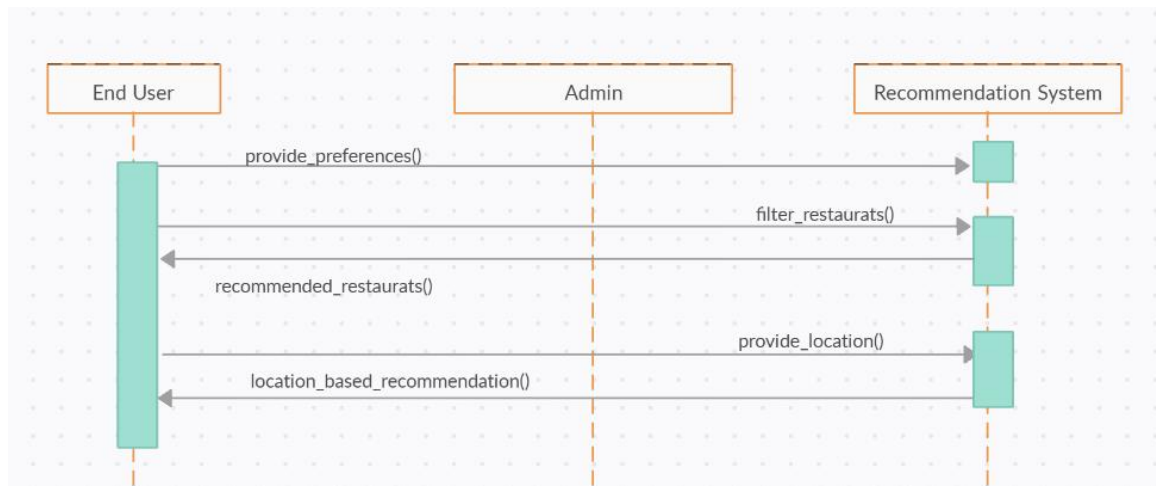


Figure 4 : Sequence Diagram for Recommendation System

3.1.4 Process Modeling

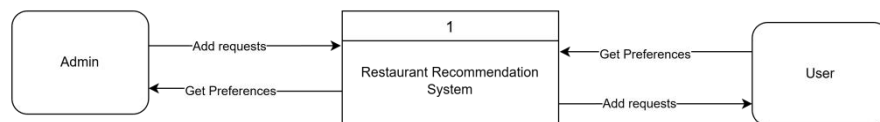


Figure 5 : DFD Level 0 for Restaurant Recommendation System

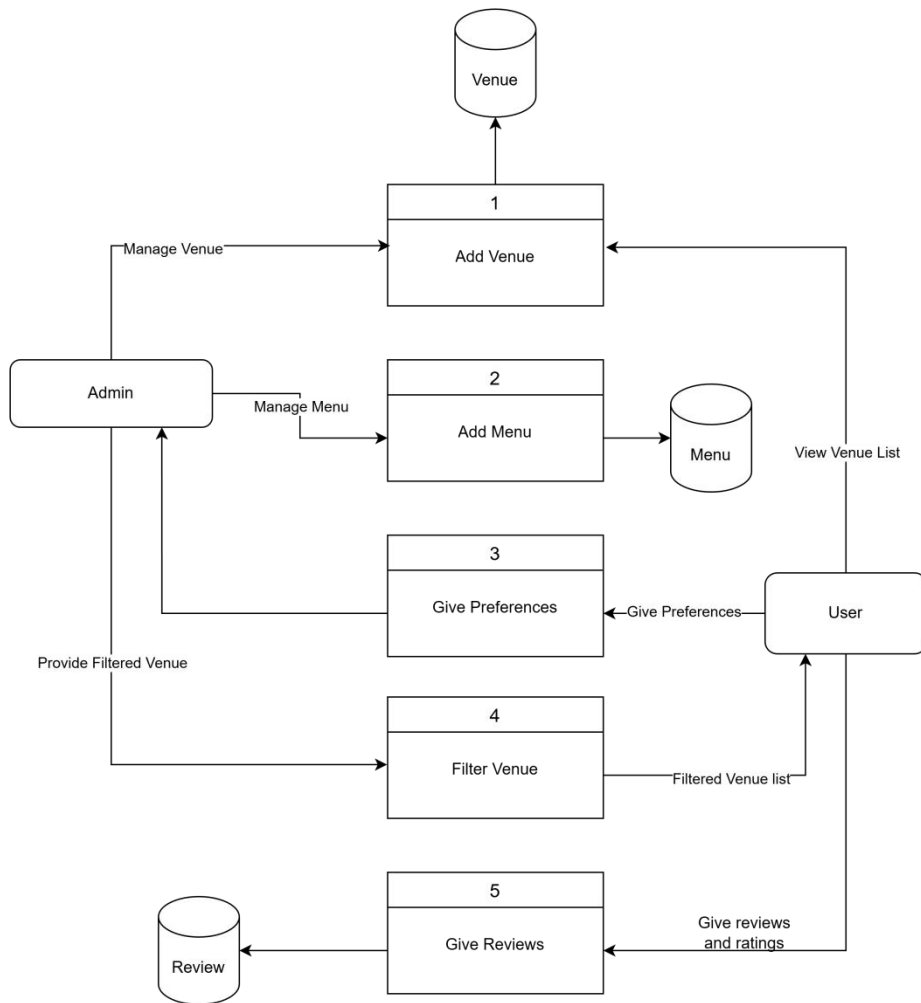


Figure 6 : DFD Level 1 for Restaurant Recommendation System

3.2 System Design

3.2.1 System Architecture

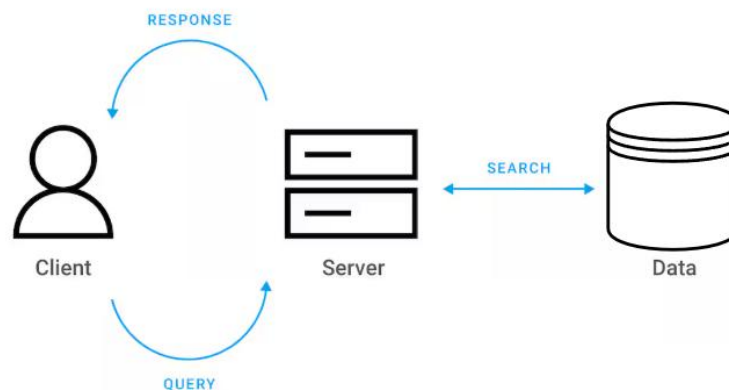


Figure 7 : Client Server Architecture

3.2.2 Database Schema Design

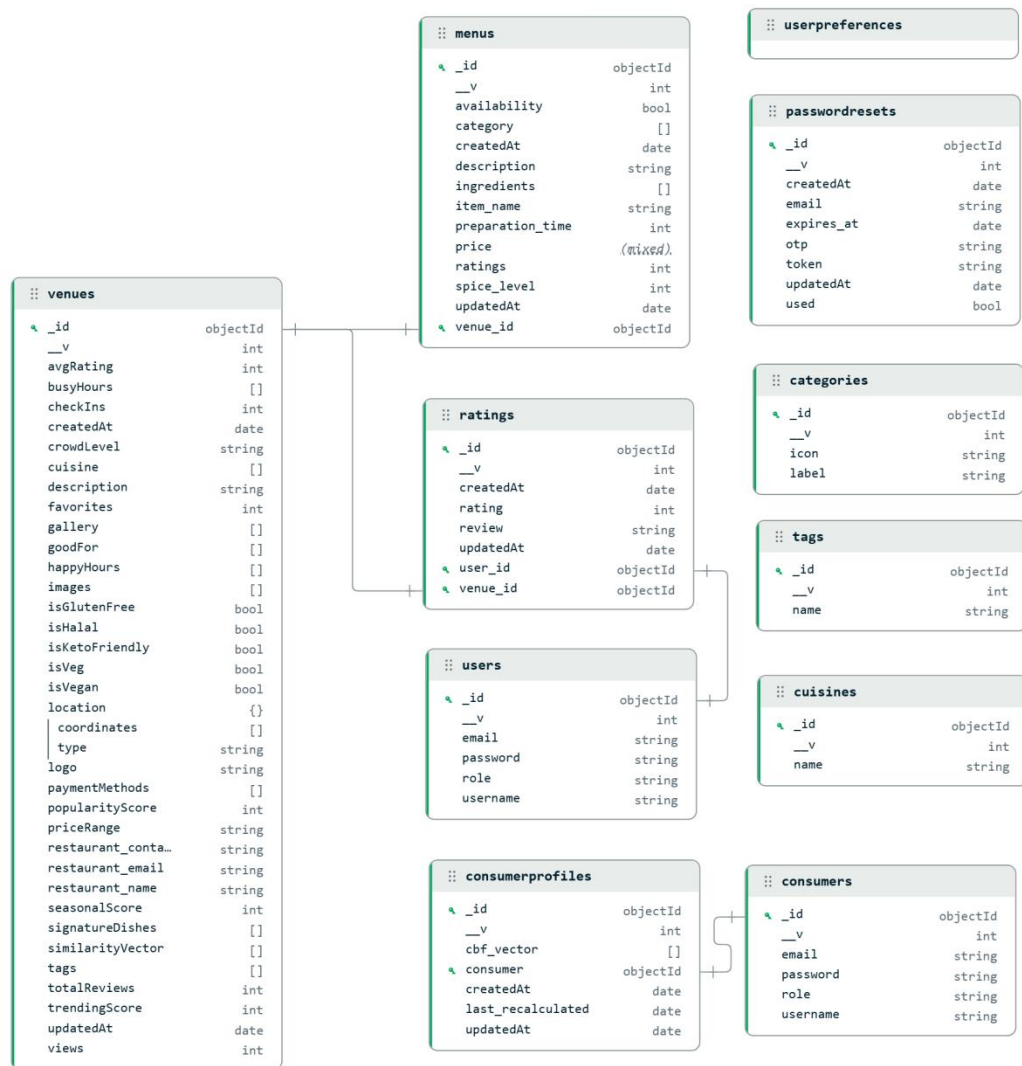


Figure. 8. Database Schema Design

3.2.3 System Flowchart

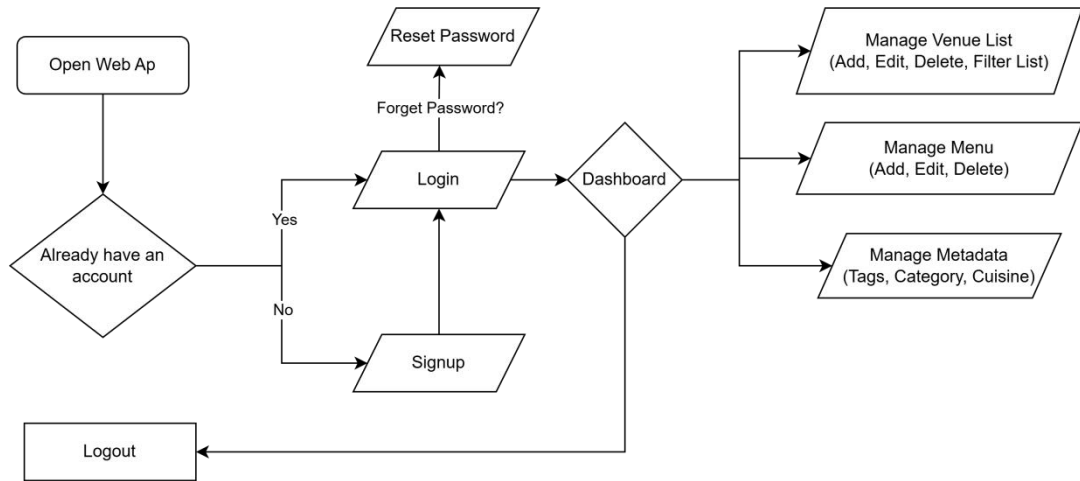


Figure. 9. Flow chart of Superadmin

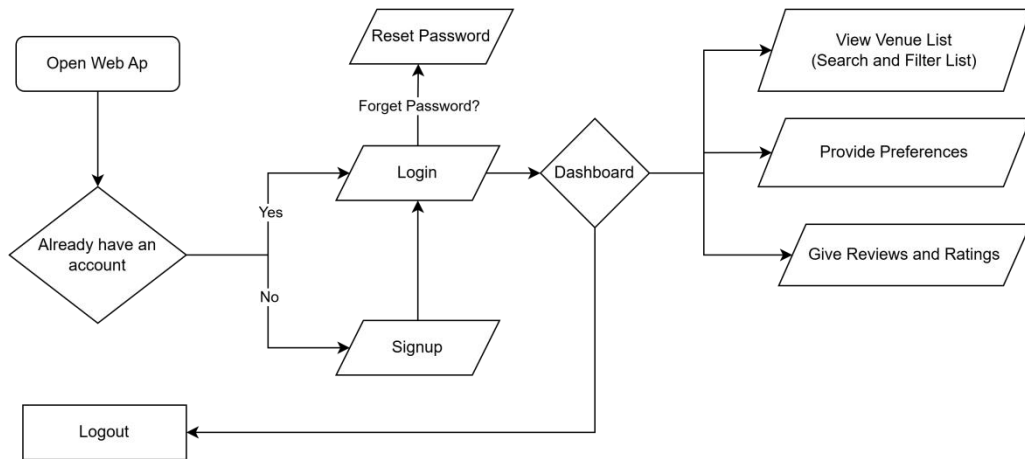


Figure. 10. Flow chart of User

3.3 Algorithm Details

The recommendation system is built using a content-based filtering approach, which focuses on analyzing the features of restaurants and recommending those that are most similar to what the user already likes. In this method, each restaurant is represented by descriptive attributes such as cuisine type, menu items, tags, and other textual information. These features are combined into a single textual representation and converted into numerical vectors using TF-IDF (Term Frequency–Inverse Document

Frequency), which measures how important each word is within the dataset. Once the restaurants are transformed into vector form, the system computes similarity between them using cosine similarity. Cosine similarity is a metric that measures the cosine of the angle between two vectors, producing a value between 0 and 1, where 1 indicates identical items and 0 indicates no similarity. This allows the system to compare restaurants based on their feature patterns rather than the length of their descriptions. After calculating similarity values, the system ranks the restaurants and recommends those with the highest similarity scores to the user's selected restaurant. This algorithm is efficient, easy to interpret, and effective for identifying restaurants with similar characteristics, making it well-suited for personalized recommendation tasks.

Cosine Similarity

Cosine similarity is the main algorithm used to measure the similarity between two restaurants or between a restaurant and a user preference vector. It calculates the cosine of the angle between two feature vectors, producing a score between 0 and 1, where a value closer to 1 indicates high similarity. Cosine similarity focuses on the direction of the vectors rather than their magnitude, making it suitable for text-based features like cuisines and tags. For two vectors AAA and BBB, the cosine similarity is defined as:

$$\text{similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \cdot \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

The core recommendation logic computes the cosine similarity between the consumer's profile vector and each restaurant's feature vector. Cosine similarity measures how closely aligned the two vectors are in direction, effectively identifying restaurants whose attributes most closely match the user's preferences, regardless of vector magnitude. Only restaurants with a positive similarity score are retained, ranked in descending order of similarity, and truncated to a configurable recommendation limit. Once the top candidates are identified, the service performs a final database lookup to retrieve full restaurant details and attaches the computed recommendation score to each result before returning them to the client. This separation of scoring and data retrieval ensures efficiency and keeps the recommendation logic independent of presentation details.

```

src > services > JS cbf-pipeline.js > ...
31  * Calculates and updates the feature vector for a single venue. Ashok Rai, 2 days ago • ui change
32  * @param {mongoose.Types.ObjectId} venueId - The ID of the newly created venue.
33  */
34  async function triggerFeatureRecalculation(venueId) {
35    try {
36      await loadCbfConfig(); // Load global config first
37
38      // 1. Fetch the raw venue data
39      const venue = await Venue.findById(venueId);
40
41      if (!venue) {
42        console.warn(`CBF: Venue ${venueId} not found.`);
43        return;
44      }
45
46      // --- A. Calculate OHE Cuisine Vector (Multi-Label) ---
47      const venueCuisines = venue.cuisine || [];
48      const cuisineVectorLength = Object.keys(GLOBAL_CUISINE_MAP).length;
49      const cbf_cuisine_vector = new Array(cuisineVectorLength).fill(0);
50
51      if (Array.isArray(venueCuisines) && venueCuisines.length > 0) {
52        venueCuisines.forEach(rawCuisineName => {
53          const cleanedCuisineName = rawCuisineName ? rawCuisineName.trim() : null;
54
55          if (cleanedCuisineName && GLOBAL_CUISINE_MAP.hasOwnProperty(cleanedCuisineName)) {
56            const index = GLOBAL_CUISINE_MAP[cleanedCuisineName];
57            cbf_cuisine_vector[index] = 1; // Set the corresponding bit to 1
58          }
59        });
60      }
61    }

```

```

c > services > JS recommendationService.js > getCBFRecommendations
21  async function getCBFRecommendations(consumerId, limit = DEFAULT_RECOMMENDATION_LIMIT) {
22
23    const userVector = consumer.cbf_vector;
24
25    const allFeatures = await RestaurantFeatures.find()
26      .select('restaurant cbf_cuisine_vector cbf_tags_vector cbf_rating_score');
27
28    if (allFeatures.length === 0) {
29      return [];
30    }
31
32    const scoredRestaurants = [];
33    const featureDimension = userVector.length;
34
35    for (const itemFeature of allFeatures) {
36      const restaurantVector = [
37        ...itemFeature.cbf_cuisine_vector,
38        ...itemFeature.cbf_tags_vector,
39        itemFeature.cbf_rating_score
40      ];
41
42      if (restaurantVector.length !== featureDimension) {
43        continue;
44      }
45
46      const similarityScore = cosineSimilarity(userVector, restaurantVector);
47
48      if (similarityScore > 0) {
49        scoredRestaurants.push({
50          restaurantId: itemFeature.restaurant,
51          score: similarityScore
52        });
53      }
54    }
55  }

```

```
// Sort and Limit
scoredRestaurants.sort((a, b) => b.score - a.score);
const topScores = scoredRestaurants.slice(0, limit);

// Final Data Retrieval
const topIds = topScores.map(item => item.restaurantId);
const recommendedVenues = await Restaurant.find({ _id: { $in: topIds } });

const finalRecommendations = recommendedVenues.map(venue => {
  const scoreData = topScores.find(s => s.restaurantId.equals(venue._id));
  return {
    ...venue.toObject(),
    recommendation_score: scoreData ? scoreData.score : 0
  };
});

return finalRecommendations;
```

```
JS consumerController.js JS consumerRoutes.js JS authController.js JS passwordResetRoutes.js JS recommendatio
src > utils > JS vector-math.js > magnitude
10 function dotProduct(vectorU, vectorR) {
11   if (vectorU.length !== vectorR.length) {
12     throw new Error("Vectors must have the same length for Dot Product calculation.");
13   }
14
15   let sum = 0;
16   for (let i = 0; i < vectorU.length; i++) {
17     sum += vectorU[i] * vectorR[i];
18   }
19   return sum;
20 }
21
22 function magnitude(vector) {
23   let sumOfSquares = 0;
24   for (let i = 0; i < vector.length; i++) {
25     sumOfSquares += vector[i] * vector[i];
26   }
27   return Math.sqrt(sumOfSquares);
28 }
29
30 function cosineSimilarity(vectorU, vectorR) {
31   const dot_product = dotProduct(vectorU, vectorR);
32   const magnitude_u = magnitude(vectorU);
33   const magnitude_r = magnitude(vectorR);
34
35   if (magnitude_u === 0 || magnitude_r === 0) {
36     return 0;
37   }
38
39   return dot_product / (magnitude_u * magnitude_r);
40 }
41
```

Haversine Algorithm

The Haversine formula is a widely used mathematical method for calculating the shortest distance between two points on the Earth's surface based on their latitude and longitude coordinates. Because the Earth is spherical rather than flat, traditional Euclidean distance calculations can lead to significant inaccuracies, particularly when distances span several kilometers or when locations are spread across different regions. The Haversine formula overcomes this limitation by incorporating trigonometric functions that account for the Earth's curvature, enabling precise measurement of real-world distances in units such as kilometers or miles.

In this restaurant recommendation system, the Haversine algorithm plays a crucial role in integrating geographical relevance into the recommendation process. The system first obtains the user's current geographic coordinates, either through device location services or user-provided data, and then applies the Haversine formula to calculate the distance between the user and each restaurant in the database. This distance information allows the system to filter out restaurants that are too far away, ensuring that only venues within a practical radius (for example, 3 km, 5 km, or 10 km) are considered. Additionally, the computed distances can be used to sort restaurants based on proximity, prioritizing those that are easier and more convenient for the user to reach.

By combining Haversine-based distance calculations with content-based similarity scores, the recommendation system achieves a balanced approach that considers both user preferences and physical accessibility. While content-based filtering ensures that recommended restaurants match the user's tastes in terms of cuisine, menu items, and tags, the Haversine algorithm ensures that these recommendations are geographically feasible. This integration significantly enhances the overall user experience by delivering restaurant suggestions that are not only personalized and relevant but also location-aware and practically usable in real-world scenarios.

$$\begin{aligned}a &= \sin^2(\Delta\phi/2) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2(\Delta\lambda/2) \\c &= 2 \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a}) \\d &= R \cdot c\end{aligned}$$

```

const haversine = (lat1, lon1, lat2, lon2, unit = 'K') => {
  // Earth's radius in different units
  const R = {
    K: 6371,    // Kilometers
    M: 3959,    // Miles
    N: 3440     // Nautical miles
  };

  You, 23 hours ago • added haversine logic

  // Convert degrees to radians
  const dLat = toRad(lat2 - lat1);
  const dLon = toRad(lon2 - lon1);
  const lat1Rad = toRad(lat1);
  const lat2Rad = toRad(lat2);

  // Haversine formula
  const a = Math.sin(dLat / 2) * Math.sin(dLat / 2) +
    Math.sin(dLon / 2) * Math.sin(dLon / 2) *
    Math.cos(lat1Rad) * Math.cos(lat2Rad);

  const c = 2 * Math.asin(Math.sqrt(a));
  const distance = R[unit] * c;

  return distance;
};

```

CHAPTER 4 - IMPLEMENTATION AND TESTING

4.1 Implementation

4.1.1 Tools used

Following are the tools and framework used for the accomplishment of this project:

Frontend Tools

- **HTML:** Used for structuring the web pages and defining elements like headings, paragraphs, forms, and images.
- **CSS:** Used to style the HTML elements, ensuring the system is visually appealing and user-friendly by controlling colors, fonts, layouts, and spacing.
- **JavaScript:** Used for client-side validation and adding dynamic, interactive elements to the system, enhancing the user experience.
- **Tailwind:** Used to create a responsive and mobile-friendly interface, ensuring the system adapts to different screen sizes.
- **React:** Used for building interactive UIs and managing dynamic content with reusable components for a seamless user experience.

Backend Tools

- **Node.js:** Used as the server-side runtime environment to handle requests, manage the system's logic, and interact with the front-end. It enables efficient handling of asynchronous operations and real-time data processing.
- **Express.js:** A web application framework built on top of Node.js that simplifies the creation of robust and scalable server-side applications. Express is used to handle routing, middleware integration, and HTTP requests, allowing for fast and efficient handling of API endpoints.

Database

- **MongoDB:** Used as the database to store and manage the system's data, including blood inventory records, donor information, and blood request details. MongoDB's NoSQL structure allows for flexible and scalable data storage, making it ideal for handling large volumes of data.

- **MongoDB Compass:** A graphical user interface for MongoDB that provides a powerful, visual tool for managing and analyzing MongoDB data. Compass allows for easy exploration of the database, real-time querying, and visualizations of data structures, helping developers and administrators quickly identify issues and optimize the database schema.

Documentation Tools

- **MS Office:** This is used for writing and editing the documentation of the system.

4.1.2 Implementation details of modules

I. Superadmin Module

1. Manage Restaurant Details : The superadmin can add, update, and delete restaurant information, including: Restaurant name, Address and location coordinates, Cuisine types, Menu items with prices, Restaurant images, and logo, Contact details

```
module.exports.create_venue = async (req, res, next) => {
  // Destructure known simple fields (name, address) and capture the rest
  const { name, address, ...otherVenueData } = req.body;
  const files = req.files;

  // A variable to hold the final, parsed data for Mongoose
  let venueDataToSave = {};

  try {
    if (!files.logo || files.logo.length === 0) {
      return res.status(400).json({
        msg: 'Logo image is required for venue creation.'
      });
    }

    const baseUrl = `${req.protocol}://${req.get("host")}`;
    const logoFilename = files.logo[0].filename;
    const logoURL = `${baseUrl}/uploads/${logoFilename}`

    let galleryURLs = [];
    if (files.images && files.images.length > 0) {
      galleryURLs = files.images.map(file => {
        return `${baseUrl}/uploads/${file.filename}`;
      });
    }
  }
}
```

2. User Management : View all registered users, Monitor user reviews

```

controllers > JS authController.js > ...
Ashok Rai, 5 months ago | 1 author (Ashok Rai)

const AdminSchema = require("../models/authModel")
module.exports.adminSignup = async(req, res)=>{
  const{email, password, role} = req.body;
  try{
    const user = await AdminSchema.create({email, password, role})

    res.status(201).json({
      msg: 'User created successfully'})
  }catch(error){
    res.status(405).json({
      msg: `Failed to create user ${error}`})
  }
}

module.exports.adminLogin = async(req, res)=>{
  try{
    const{username, password} = req.body;
    const loginSuccess = await AdminSchema.adminLogin
  }catch(error){
    res.status(403).json({
      msg: `Failed to login user: ${error}`
    })
  }
}
}

```

3. Dashboard and Analytic : Track number of restaurants registered, Monitor user review statistics, View trending cuisines and popular restaurants, Analyze platform performance through graphs and reports

II. User Module

1. User Registration and Login : Users can create an account, Secure login with password encryption, Profile creation including name, preferences, and optional profile image.

```
const createToken = (id) => {  
    return jwt.sign({ id }, "token id", { expiresIn: maxAge });  
};  
  
// @desc Login user  
module.exports.loginUser = async (req, res, next) => {  
    const { email, password } = req.body;  
    console.log("login>>>>>>>>>>>>>>>>>>", email, password)  
  
    try {  
        const user = await Consumer.login(email, password);  
        const token = createToken(user._id);  
  
        res.cookie("jwt", token, { httpOnly: true, maxAge: maxAge * 1000 });  
        res.status(200).json({  
            success:true,  
            message: "Successfully logged in.",  
            user,  
            token,  
        });  
    } catch (error) {  
        // const errors = handleError(error);  
        // res.status(400).json({ errors });  
        next(error)  
    }  
};
```

2. View Restaurant Listings : Browse all restaurants with key details (name, cuisine, rating, location), Access full restaurant profile including menu, images, facilities, opening hours, Search restaurants by keywords, location, cuisines, or price range, Apply filters (distance, rating, cuisine type, categories, etc.).

3. Restaurant Recommendation System: Get personalized recommendations based on:

- User preferences
- Popular and trending restaurants nearby
- Smart sorting options (Top Rated, Nearby, Budget Friendly, Cuisine Type).

4. Review and Rating System : Users can write reviews and give ratings to restaurants they have visited, Edit or delete their own reviews, View reviews from other user.

```

module.exports.all_review = async (req, res, next) => {
  try{
    const allReviews = await ReviewSchema.find();
    return res.status(200).json({
      count: allReviews.length, data: allReviews
    })
  }catch(error){
    next(error);
  }
}

module.exports.get_venue_reviews = async (req, res, next) =>
};

module.exports.create_review = async (req, res, next) => {
  const newData = req.body;
  console.log(newData, "soalr.....")
  try{
    const review = await ReviewSchema.create(newData);
    res.status(201).json({
      msg: "Review created successfully.",
      data: review,
      success: true
    })
  }catch(error){
    next(error)
  }
}

```

5. Restaurant Interaction Features : Check menus and estimated pricing.

```

module.exports.get_single_menu = async (req, res, next) =>{
  const id = req.params.id;
  try{
    const singleData = await MenuSchema.findById(id);
    res.status(200).json({
      success: true,
      data: singleData,
      message: "Menu fetched successfully."
    })
    if(!singleData){
      res.status(404).json({
        success: false,
        data: '',
        message: "Menu item not found."
      })
    }
  }catch(error){
    next(error)
  }
}

module.exports.get_menu_by_restaurant = async (req, res, next) => {
  const id = req.params.id;

  try {
    const restaurantMenu = await MenuSchema.find({ venue_id: id });

    if (!restaurantMenu || restaurantMenu.length === 0) {
      return res.status(404).json({
        success: false,
        data: [],
        message: "No menu items found for this restaurant."
      });
    }

    return res.status(200).json({
      success: true
    })
  }
}

```

6. Location-Based Services: View nearby restaurants using map integration, Detect current location to show closest dining options.

4.2 Testing

4.1.1 Test Cases for Unit Testing

Table 2 : Test case for User Registration

S.No	Test Name	Input	Expected Output	Actual Output	Test Result
1	Registration	Register as a new user: Name = Shraddha Dongol Username = shraddha Email = shraddha@gmail.com Password = shraddha@123 Role = Admin Auth token = admin	New user created	New user created	Pass
2	Registration	Register as a new user: Name = Ashok Rai Username = ashok Email = ashok.com Password = ashok@123 Role = User	Invalid email address	Invalid email address	Pass

Table 3 : Test case for User Login

S.No.	Test Name	Input	Expected Output	Actual Output	Test Result
1	Login	Login: Username = shraddha Password = shraddha@123	Username and password authenticated and dashboard opened	Username and password authenticated and dashboard opened	Pass
2	Login	Login: Username = shraddha Password = shraddha23	Incorrect credentials and error message	Incorrect credentials and error message	Pass

4.2.2 Test cases for System Testing

Table 4 : Test case for Filtering Restaurants

S.No.	Test Name	Input	Expected Output	Actual Output	Test Result
1	Filter by Location	User grants location permission	System retrieves latitude and longitude coordinates and identifies the user's location	Displays a list of restaurants near the user's current location	Pass
2	Filter by different options	User provides selected filter criteria	System filters the list of restaurants based on the provided criteria	Displays a list of restaurants that meet the selected filters	Pass
3	Search & Filter Combined	User enters keyword (e.g., "Pizza") and applies filters	System applies both keyword search and filters	Displays restaurants matching keyword and filter criteria	Pass
4	Invalid Location	User denies location access	System handles missing location data gracefully	Prompts user to enter location manually or shows all restaurants	Pass
5	No Matching Results	User selects a combination of filters with no matching restaurants	System checks filters and handles empty results	System checks filters and handles empty results	Pass
6	Reset Filters	User resets all filter options	System clears all applied filters	Displays all restaurants without any filter	Pass

Table 5 : Test case for Managing Restaurants

S.No.	Test Name	Input	Expected Output	Actual Output	Test Result
1	Create New Restaurant	SuperAdmin enters restaurant name, address, phone, category, tags, location, cuisines	System saves the restaurant and displays confirmation	Restaurant added successfully and visible in the list	Pass
2	Edit Restaurant Details	SuperAdmin updates restaurant details (name, address, tags, etc.)	System saves changes and updates the restaurant information	Updated restaurant info displayed correctly	Pass
3	Delete Restaurant	Superadmin selects a restaurant and deletes it	System removes restaurant from database after confirmation	Restaurant no longer visible in the list	Pass
4	Create Restaurant with Missing Fields	Superadmin leaves required fields blank (e.g., name or address)	System shows validation error	“Restaurant name/address is required”	Pass

Table 6 : Test case for Managing Menu

S.No.	Test Name	Input	Expected Output	Actual Output	Test Result
1	Create Menu for Restaurant	Superadmin adds menu items with name, description, and price	System saves menu items under the correct restaurant	Menu items displayed under the restaurant	Pass
2	Add Menu Item with Missing Fields	Superadmin leaves name or price blank	System shows validation error	Validation message shown - Menu name and price are required	Pass
3	Edit Menu Item	SuperAdmin updates menu item details	System updates the menu item successfully	Updated menu displayed under the restaurant	Pass

4.3 Result Analysis

The results of the testing process indicate that all core system functionalities including user registration, user login, and restaurant filtering are operating as intended. The registration module successfully created new user accounts when valid information was provided and accurately detected invalid inputs, such as an incorrect email format. The login module consistently authenticated users with correct credentials while preventing access and displaying appropriate error messages when incorrect details were entered. Similarly, the restaurant filtering feature performed effectively, retrieving the user's location when permission was granted and accurately filtering restaurants based on selected criteria. Across all test cases, the actual outputs matched the expected results, demonstrating that the system is stable, reliable, and meets the defined functional requirements.

CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS

5.1 Conclusion

In conclusion, the development of this restaurant recommendation system aims to bridge the gap between users seeking the perfect place to eat or celebrate special moments and restaurants aiming to attract the right audience. By leveraging modern web technologies like React and Node.js, the system will deliver a responsive, scalable, and accessible solution tailored to user preferences.

The use of Agile methodology will ensure that development is iterative, collaborative, and adaptive to feedback, allowing for continuous improvement throughout the project lifecycle. Emphasizing both functional and non-functional requirements ensures the system is not only feature-rich but also secure, fast, and user-friendly.

This project has the potential to improve the restaurant discovery experience for users while also providing a digital platform for restaurant owners to manage and showcase their services. With thoughtful design, user-centric functionality, and scalable architecture, the system is expected to make a significant impact on the way users explore dining experiences.

5.2 Future Recommendation

- Include real-time information such as table availability or peak hours.
- Add features for table booking or food delivery integration.
- Create a mobile app for easier access on phones.
- Improve review verification to reduce fake reviews.
- Add a dashboard for restaurant owners to manage their menu, images, and offers.
- Support multiple languages for better accessibility.

REFERENCE

- [1] Zomato – www.zomato.com, Accessed: May 2025.
- [2] Y. Park, Y. Tuzhilin, “The long tail of recommender systems and how to leverage it,” *Proceedings of the 2008 ACM Conference on Recommender Systems*, pp. 11-18, 2008.
- [3] V. W. Zheng, Y. Zheng, X. Xie, Q. Yang, “Collaborative location and activity recommendations with GPS history data,” *Proceedings of the 19th international conference on World Wide Web*, pp. 1029–1038, 2010.
- [4] F. Liu, L. Yu, X. Yang, "Context-Aware Restaurant Recommendation for Mobile Users," *International Conference on Web-Age Information Management*, Springer, 2014.
- [5] M. Chen, X. Xu, M. Wang, “Personalized Restaurant Recommendation System Based on Deep Learning,” *IEEE Access*, vol. 6, pp. 69087–69096, 2018.
- [6] Yelp Inc., “Yelp Open Dataset,” <https://www.yelp.com/dataset>, accessed 2025.
- [7] R. He, W.-C. Kang, and J. McAuley, “Translation-based recommendation,” in *Proc. 11th ACM Conf. Recommender Systems (RecSys)*, 2017, pp. 161–169.

APPENDIX

Welcome

Sign up for a new RestroNet account

Email

Fullname

Phone Number

Password



Confirm Password

Sign Up

Already have an account? [Login](#)

Welcome back

Login to your RestroNet account

Email

Password



Login

Don't have an account? [Sign up](#)



Welcome! Let's personalize your experience

Tell us what you love, and we'll recommend the perfect restaurants for you

Step 1 of 4

What cuisines do you enjoy?

Select all that apply

Italian	Chinese	Japanese	Mexican
Indian	✓ Nepalese	Thai	Mediterranean
American	Korean	French	Vietnamese
Spanish	Middle Eastern		

Back

Continue

Your Preferences:

Italian Japanese Nepalese Dinner Lunch Romantic Outdoor Seating Within 5 km

Update Your Preferences

Showing 6 recommended restaurants for you.



Baber Mahal Bistro

A boutique-style bistro offering fusion dishes, peaceful ambiance, and an attached art gallery for a unique dining experience.

★★★★★

@ babarmahal

Nepalese Continental Chinese

View Details



Lakeview Coffee Corner

A peaceful café overlooking Phewa Lake, perfect for morning coffee, light meals, and sunset views.

★★★★★

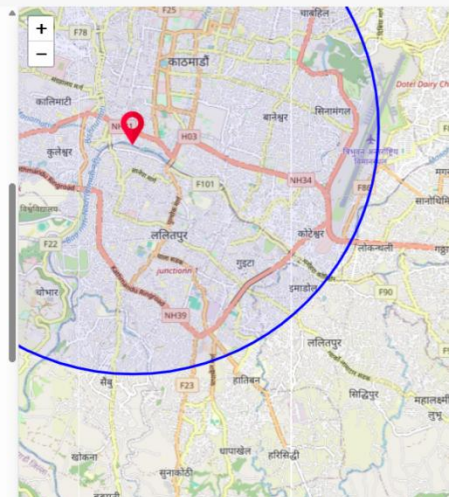
@ new road pokhara

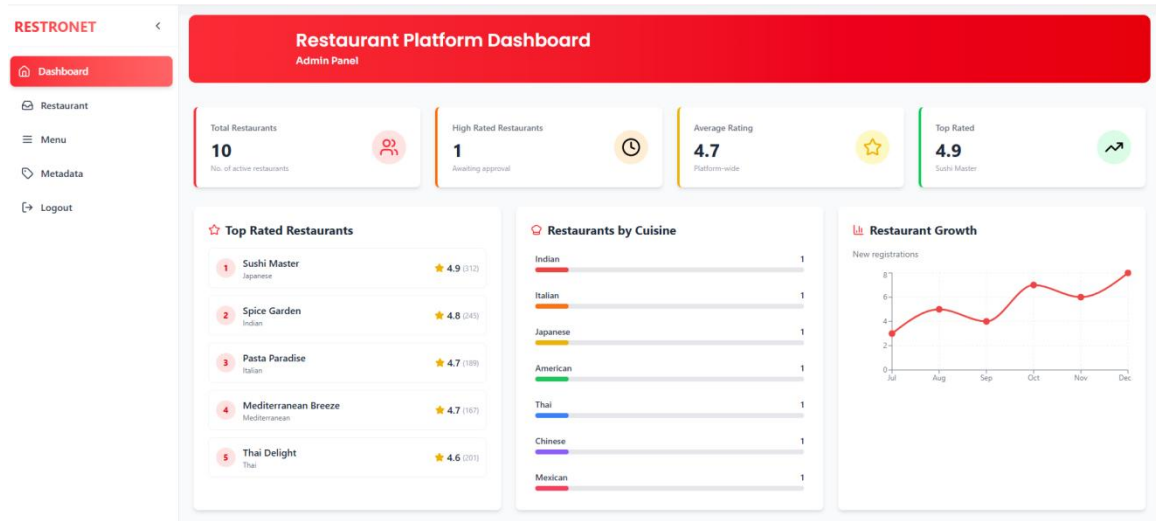
View Details



Nepali Coffee Café

A select blend of coffee, offering specialty brews, freshly brewed coffee, and sustainable meals.





RESTRONET <

Dashboard
Restaurant
Menu
Metadata
Logout

Restaurant Management
Manage your restaurant listings and details

+ Add New Restaurant

Search restaurants...

LOGO	NAME	LOCATION	CONTACT	CUISINE	MENU MANAGER	ACTIONS
	Himalayan Brew House Updated	Jhamsikhel	9812345678	Various	Manage	View Edit Delete
	Nepali Thakali Kitchen	thamel	9812345678	Various	Manage	View Edit Delete
	Baber Mahal Bistro	babarmahal	9812345678	Various	Manage	View Edit Delete
	Lakeview Coffee Corner	new road pokhara	9812345678	Various	Manage	View Edit Delete
	Nepali Coffee Café	dillibazar	9800000000	Various	Manage	View Edit Delete
	Achar Ghar Thakali	kamalpokhari	9800000000	Various	Manage	View Edit Delete

Rows per page: 10 1-6 of 6 < > >>

Update Restaurant

Restaurant Details
Fill the form to register the restaurant

Restaurant Name *

Achar Ghar Thakali

Location *

kamalpokhari

Phone Number *

9800000000

Email *

rcwakommu@mailinator.com

Description

An Experiential Nepali Cuisine Restaurant With variation of Dal-Bhat & Pickles - Khalpi Pickle brings the perfect balance of crunch and tang, helping wit

Restaurant Logo
Upload Image

Drag & drop or click to upload

Selected logo: Existing logo

Cuisine *

Nepalese X Indian X

Tags *

chefs special X Spicy X peaceful X

RESTRONET

Dashboard

Restaurant

Menu

Metadata

Logout

Menu Management

Manage your menu listings and details

Basic Information

Select Restaurant

Select restaurant

Item Name *

e.g., Margherita Pizza

Description

Brief description of the item...

Price (\$) *

0.00

Prep Time (minutes)

20

Ingredients & Specifications

Ingredients *

Flour, Tomato, Mozzarella, Basil (comma separated)

Separate ingredients with commas

Spice Level (0-5)

Available for order

Categories & Tags

Category *

Breakfast

Tags

Select cuisines

Upload Images

Drag & drop or click to upload

Add Item

RESTRONET

Dashboard

Restaurant

Menu

Metadata

Logout

Restaurant Metadata Manager

Categories

Cuisines

Tags

Add New Category

Category *

Enter Category

Category Icon *

Upload Image

Drag & drop or click to upload

Submit

List of items

Total Categories : 5

Breakfast

Brunch

Lunch